

Tutorial-6

1. The following sequence of the operations is performed on a stack PUSH(10), PUSH(20), POP, PUSH(10), PUSH(20), POP, POP, POP, PUSH(20), POP the sequence of values popped out is

Ans: 20,20,10,10,20

2. What is the output printed by the following program.

```
#include <stdio.h>
int *A, stkTop;
int stkFunc (int opcode, int val)
{
    static int size=0, stkTop=0;
    switch (opcode)
    {
        case -1:
            size = val;
            break;
        case 0:
            if (stkTop < size ) A[stkTop++]=val;
            break;
        default:
            if (stkTop) return A[--stkTop];
    }
    return -1;
}
int main()
{
    int B[20];
    A=B; stkTop = -1;
    stkFunc (-1, 10);
    stkFunc (0, 5);
    stkFunc (0, 10);
    printf ("%d\n", stkFunc(1, 0)+ stkFunc(1, 0));
}
```

Explanation: The code in main, basically initializes a stack of size 10, then pushes 5, then pushes 10.

Finally the printf statement prints sum of two pop operations which is $10 + 5 = 15$.

stkFunc (-1, 10); // Initialize size as 10

```
stkFunc (0, 5); // push 5
stkFunc (0, 10); // push 10 // print sum of two pop
printf ("%d\n", stkFunc(1, 0) + stkFunc(1, 0));
```

2. A single array A[1..MAXSIZE] is used to implement two stacks. The two stacks grow from opposite ends of the array. Variables top1 and top2 (top1 < top2) point to the location of the topmost element in each of the stacks. If the space is to be used efficiently, the condition for “stack full” is

- a) (top1 = MAXSIZE/2) and (top2 = MAXSIZE/2+1)
- b) top1 + top2 = MAXSIZE
- c) (top1 = MAXSIZE/2) or (top2 = MAXSIZE)
- d) top1 = top2 - 1

Answer(d)

If we are to use space efficiently then size of the any stack can be more than MAXSIZE/2. Both stacks will grow from both ends and if any of the stack top reaches near to the other top then stacks are full. So the condition will be top1 = top2 - 1 (given that top1 < top2)

4. The following postfix expression with single digit operands is evaluated using a stack:

8 2 3 ^ / 2 3 * + 5 1 * -

The top two elements of the stack after the first * is evaluated are:

Ans: 6,1

- 1. First three tokens are values, so they are simply pushed. stack has 8,2,3
- 2. When ^ is read, top two are popped and power(2^3) is calculated- stack has 8,8
- 3. When / is read, top two are popped and division(8/8) is performed- stack has 1
- 4. Next two tokens are values, so they are simply pushed. - stack has 1, 2,3
- 5. When * comes, top two are popped and multiplication is performed- stack 1,

- 5) The value of the following postfix expression

5 4 6 + * 4 9 3 / + * is

Ans: 350

- 6) The postfix form of the expression (A+B)*(C*D-E)*F/G?

- AB+ CD*E – FG /**

Symbol	stack	postfix string
(	(	empty
A	(	A
+	(+	A
B	(+	AB
)	empty	AB +
*	*	AB +
(	* (	AB +
C	* (	AB + C
*	* (*	AB + C
D	* (*	AB + CD
-	* (-	AB + CD *
E	* (-	AB + CD * E
)	*	AB + CD * E -
*	**	AB + CD * E -
F	**	AB + CD * E - F
/	** /	AB + CD * E - F
G	** /	AB + CD * E - FG
	Empty	AB + CD * E - FG / **

Q.7

If we have six stack operations pushing and popping each of A, B and C such that push(A) must occur before push(B) which must occur before push(C), then A, C, B is a possible order for the pop operations, since this could be our sequence : push(A), pop(A), push(B), push(C), pop(C), pop(B).

Which one of the following orders could not be the order the pop operations are run, if we are to satisfy the requirements described above ?

- A** ABC
- B** CBA
- C** BAC
- D** CAB

Ans: D

Solution: <https://academyera.com/ugc-net-cs-2010-june-paper-2-queues-and-stacks>

Q.8 Sort a Stack

Given a stack, the task is to sort it such that the top of the stack has the greatest element.

Example 1:

Input:

Stack: 3 2 1

Output: 3 2 1

Example 2:

Input:

Stack: 11 2 32 3 41

Output: 41 32 11 3 2

Your Task:

You don't have to read input or print anything. Your task is to complete the function **sort()** which sorts the elements present in the given stack.

Amazon

Goldman Sachs

IBM

Intuit

Kuliza

Microsoft

Yahoo

Solution: <https://www.youtube.com/watch?v=xivWbCBPEdc>

Weblink: <https://www.geeksforgeeks.org/sort-a-stack-using-recursion/>

Sorting of stack element:

```
sortStack(stack S)
```

```
    if stack is not empty:
```

```
        temp = pop(S);
```

```
        sortStack(S);
```

```
        sortedInsert(S, temp);
```

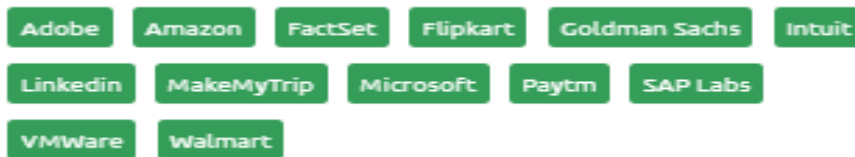
```

sortedInsert(Stack S, element)
    if (stack is empty || element > top element)
        push(S, elem)
    else
        temp = pop(S)
        sortedInsert(S, element)
        push(S, temp)

```

Q.9 Special Stack

Design a Data Structure SpecialStack that supports all the stack operations like push(), pop(), isEmpty(), isFull() and an additional operation getMin() which should return minimum element from the SpecialStack.



Example 1:

Input:

Stack: 18 19 29 15 16

Output: 15

Explanation:

The minimum element of the stack is 15.

Your Task:

Since this is a function problem, you don't need to take inputs. You just have to complete 5 functions, **push()** which takes the stack and an integer x as input and pushes it into the stack; **pop()** which takes the stack as input and pops out the topmost element from the stack; **isEmpty()** which takes the stack as input and returns true/false depending upon whether the stack is empty or not; **isFull()** which takes the stack and the size of the stack as input and returns true/false depending upon whether the stack is full or not (depending upon the

given size); **getMin()** which takes the stack as input and returns the minimum element of the stack.

Note: The output of the code will be the value returned by **getMin()** function.

Solution: <https://www.youtube.com/watch?v=gd9xEAnxXzc>

Video: <https://www.youtube.com/watch?v=QMIDCR9xyd8>

Weblink: <https://www.geeksforgeeks.org/sort-a-stack-using-recursion/>

Q.10

Given an expression string **x**. Examine whether the pairs and the orders of “{“,”}”,“(“,”)”, “[“,”]” are correct in exp.

For example, the function should return 'true' for exp = “[()]{}{[()()]()}" and 'false' for exp = “[()]”.



Example 1:

Input:

{([])}

Output:

true

Explanation:

{ ([]) }. Same colored brackets can form balanced pairs, with 0 number of unbalanced bracket.

Example 2:

Input:

()

Output:

true

Explanation:

(). Same bracket can form balanced pairs,
and here only 1 type of bracket is
present and in balanced way.

Example 3:

Input:

([]

Output:

false

Explanation:

([] . Here square bracket is balanced but
the small bracket is not balanced and
Hence , the output will be unbalanced.

Video Link: <https://www.youtube.com/watch?v=FPvAfpSXjRw>

Q.11 Queue using two Stacks



Q.12 Reverse individual words



Given a string str, we need to print reverse of individual words.

Examples:

Input : Hello World

Output : olleH dlroW

Input : Geeks for Geeks

Output : skeeG rof skeeG

Method 1 (Simple): Generate all words separated by space. One by one reverse words and print them separated by space.

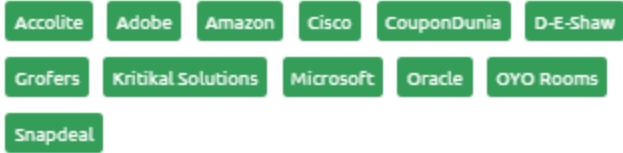
Method 2 (Space Efficient): We use a stack to push all words before space. As soon as we encounter a space, we empty the stack.

Solution:

Video Link: <https://www.youtube.com/watch?v=5eLHglr811U>

https://www.youtube.com/watch?v=sXFtdX5Qn_k

Q.13 Stack using two queues



Implement a Stack using two queues **q1** and **q2**.

Example 1:

Input:

push(2)

push(3)

pop()

push(4)

pop()

Output: 3 4

Explanation:

push(2) the stack will be {2}

push(3) the stack will be {2 3}

pop() popped element will be 3 the
stack will be {2}

push(4) the stack will be {2 4}

pop() popped element will be 4

Example 2:

Input:

push(2)

pop()

pop()

push(3)

Output: 2 -1

Your Task:

Since this is a function problem, you don't need to take inputs. You are required to complete the two methods **push()** which takes an integer 'x' as input denoting the element to be pushed into the stack and **pop()** which returns the integer popped out from the stack(-1 if the stack is empty).

Solution:

Video link: <https://www.youtube.com/watch?v=ww5Ac232WEU>

Q.14 Queue using two Stacks

implement a Queue using 2 stacks **s1** and **s2** .

A Query **Q** is of 2 Types

- (i) 1 x (a query of this type means pushing 'x' into the queue)
- (ii) 2 (a query of this type means to pop element from queue and print the popped element)



Example 1:

Input:

5

1 2 1 3 2 1 4 2

Output:

2 3

Explanation:

In the first testcase

1 2 the queue will be {2}

1 3 the queue will be {2 3}

2 popped element will be 2 the queue
will be {3}
1 4 the queue will be {3 4}
2 popped element will be 3.

Example 2:

Input:

4
1 2 2 2 1 4

Output:

2 -1

Explanation:

In the second testcase

1 2 the queue will be {2}

2 popped element will be 2 and
then the queue will be empty

2 the queue is empty and hence -1

1 4 the queue will be {4}.

Solution: <https://www.youtube.com/watch?v=n1nsfg8O4Mk>

Q.15

Consider a standard Circular Queue 'q' implementation (which has the same condition for Queue Full and Queue Empty) whose size is 11 and the elements of the queue are

q[0], q[1], q[2].....q[10].

The front and rear pointers are initialized to point at q[2] . In which position will the ninth element be added?

- A** Q[0]
- B** Q[1]
- C** Q[9]
- D** Q[10]

Solution: A (ISRO 2014 question)

In circular queue

- Front = Rear then queue is empty
- $(Rear+1) \bmod n == Front$ then queue is full
- For enqueue, we increment the REAR pointer and then insert an element
- For dequeue, we increment the FRONT and then remove the element at that position.

1st element inserted at q[3]

2nd element at q[4]

3rd element at q[5]

and so on so 9th element inserted at q[0]

Front and Rear pointers are initialized to point at q[2]

